

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для задачи «исключающее ИЛИ» с числовыми классами: сравнение функций потерь MSE и Binary Cross-Entropy»

Выполнила:

Студент 4 курса

Группы ИИ-26

Данилюк Т.В.

Проверил:

Никитин С.К.

Цель работы: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 из Лабораторной работы №1.
 2. Поскольку BCE определена для выходов в диапазоне $(0; 1)$, а целевые значения в вашем варианте – произвольные числа c_0 и c_1 из диапазона $[-10; 10]$, реализовать нормализацию (масштабирование) целевых значений перед обучением и обратное преобразование выхода сети после обучения:
 - привести c_0 и c_1 к бинарным меткам 0 и 1;
 - обучать сеть на нормализованных целевых значениях с функцией потерь BCE;
 - при выводе результата преобразовывать выход сети $\hat{y} \in (0; 1)$ обратно в шкалу $[c_0; c_1]$.
 3. Реализовать корректный градиент для связки BCE + сигмоида на выходном слое; для скрытого слоя градиент передаётся по стандартному правилу цепочки.
 4. Обучить сеть на полной таблице истинности своего варианта (4 примера) в двух конфигурациях:
 - **Конфигурация А (базовая, из ЛР №1):** MLP 2-2-1, функция потерь MSE, целевые значения – исходные c_0/c_1 (без нормализации), выходной слой – сигмоида. Результаты можно взять из отчёта по ЛР №1 или, при необходимости, повторить обучение с фиксацией всех метрик.
 - **Конфигурация Б (новая):** MLP 2-2-1, функция потерь BCE, целевые значения – нормализованные $(0/1)$, с последующим пересчётом выхода в шкалу $[c_0; c_1]$.
- Критерий остановки для обеих конфигураций одинаков – суммарная ошибка $E_s \leq E_e$ (используйте то же значение E_e , что и в ЛР №1, либо согласуйте новое значение отдельно для MSE и для BCE, поскольку это разные по своей природе величины – обоснуйте выбор в отчёте).
5. Для каждой конфигурации зафиксировать:
 - количество эпох до достижения критерия остановки (либо факт недостижения за максимально допустимое число эпох);
 - итоговую суммарную ошибку;
 - ассигасу на всех 4 примерах таблицы истинности;
 - среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
 6. Провести не менее 5 запусков обучения для каждой конфигурации с разной случайной инициализацией весов (менять seed) и сравнить устойчивость сходимости: в скольких из 5 запусков сеть достигла критерия остановки, и какой разброс по числу эпох наблюдается.
 7. Построить общий график зависимости суммарной ошибки от номера эпохи для конфигурации А (MSE) и конфигурации Б (BCE) на одних осях (для представительного запуска каждой конфигурации).
 8. Реализовать блок визуализации результатов, включающий:
 - **График сходимости** – суммарная ошибка E_s от номера эпохи, конфигурации А и Б на одних осях, разными цветами/стилями линий, с обозначением порога E_e .
 - **Диаграмму разброса числа эпох по 5 запускам** (bar chart или scatter) для конфигурации А и конфигурации Б – наглядно показывающую устойчивость сходимости из п. 6 (например, столбики числа эпох для каждого из 5 seed, сгруппированные по конфигурации; запуски, не достигшие критерия остановки, отметить отдельно, например штриховкой или отдельным цветом).
 - **Визуализацию разделяющей поверхности скрытого слоя** – для представительного обученного экземпляра каждой конфигурации построить на плоскости (А, В) карту выходного значения сети (heatmap или contour plot) с наложенными точками обучающей выборки (4 примера таблицы истинности), закрашенными по истинному классу. Диапазон осей А и В – $[-10; 10]$ (весь допустимый диапазон входных значений). Для конфигурации Б цветовую шкалу можно строить как по нормализованному $\hat{y} \in (0; 1)$, так и по пересчитанному значению в шкале $[c_0; c_1]$ – выбрать более наглядный вариант и обосновать выбор.
 - **Диаграмму сравнения точности восстановления шкалы** – столбчатую диаграмму средней абсолютной ошибки для конфигурации А и конфигурации Б, чтобы наглядно сопоставить точность регрессии к исходной шкале $[c_0; c_1]$.

Каждый график должен иметь подписанные оси, заголовок и легенду (где применимо). Разделяющую поверхность и графики точности рекомендуется разместить рядом (например, subplot 2×2 или отдельными рисунками) для удобства сравнения в отчёте.

9. Реализовать режим функционирования сети: пользователь вводит пару чисел (A, B) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1). Сеть должна выводить:

- выход в нормализованной шкале $\hat{y} \in (0; 1)$ (с точностью до 4 знаков);
- выход, пересчитанный в исходную шкалу $[c_0; c_1]$ по формуле из п. 2;
- к какому классу (c_0 или c_1) ближе полученное значение.

Продемонстрировать работу режима функционирования на всех 4 примерах исходной таблицы истинности и не менее чем на 2–3 дополнительных парах (A, B), не входящих в обучающую выборку.

Вариант 2 ($c_0 = 2, c_1 = 10$)

A	B	$A \oplus B$
2	2	2
2	10	10
10	2	10
10	10	2

Описание изменений в архитектуре и алгоритме обучения

1. Формула нормализации целевых значений

Поскольку BCE определена для выходов в диапазоне (0; 1), а целевые значения — произвольные числа c_0 и c_1 , необходимо привести их к бинарным меткам:

$$y_{norm} = \frac{y - c_{min}}{c_{max} - c_{min}}$$

где $c_{min} = 2, c_{max} = 10$

2. Формула обратного пересчёта выхода

После обучения выход сети $\in (0; 1)$ преобразуется обратно в шкалу $[c_0; c_1]$:

$$y_{real} = (c_{max} - c_{min}) \cdot y + c_{min}$$

3. Вывод градиента для BCE + сигмоида

Функция потерь BCE:

$$L = -[y \cdot \ln(\hat{y}) + (1 - y) \cdot \ln(1 - \hat{y})]$$

Производная BCE по выходу :

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}$$

Производная сигмоиды:

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$$

Итоговый градиент для выходного слоя (по правилу цепочки):

$$\delta_2 = \frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1 - \hat{y}) = \hat{y} - y$$

Ключевое отличие: В коде используется $\delta_2 = y - \hat{y}$ (с обратным знаком), так как обновление весов происходит по правилу: $W = W + \eta \cdot \delta$

Таким образом, градиент связки BCE + Sigmoid упрощается до $(y - \hat{y})$, что исключает множитель $\hat{y}(1 - \hat{y})$ и предотвращает затухание градиента при больших ошибках.

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
c0 = 2.0
c1 = 10.0
```

```
TARGET_ERROR_MSE = 0.01
TARGET_ERROR_BCE = 0.05
MAX_EPOCHS = 10000
LEARNING_RATE = 0.5
```

```
# =====
# Данные
```

```

# =====
X_raw = np.array([[C0, C0], [C0, C1], [C1, C0], [C1, C1]])
y_raw = np.array([C0, C1, C1, C0]).reshape(-1, 1)

# =====
# Вспомогательные функции
# =====
def normalize(val, c_min=C0, c_max=C1):
    return (val - c_min) / (c_max - c_min)

def denormalize(val, c_min=C0, c_max=C1):
    return val * (c_max - c_min) + c_min

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -250, 250)))

def sigmoid_derivative(s):
    return s * (1.0 - s)

X_norm = normalize(X_raw)
y_norm = normalize(y_raw)

# =====
# Класс MLP с функцией потерь MSE
# =====
class MLP_MSE:
    def __init__(self, lr, seed=None):
        if seed is not None:
            np.random.seed(seed)
        self.lr = lr
        self.W1 = np.random.randn(2, 2) * 0.5
        self.b1 = np.random.randn(1, 2) * 0.5
        self.W2 = np.random.randn(2, 1) * 0.5
        self.b2 = np.random.randn(1, 1) * 0.5

    def forward(self, x):
        self.z1 = np.dot(x, self.W1) + self.b1
        self.a1 = sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = sigmoid(self.z2)
        return self.a2

    def train_step(self, x, y_true):
        pred = self.forward(x)
        error = y_true - pred

        delta2 = error * sigmoid_derivative(pred)
        delta1 = np.dot(delta2, self.W2.T) * sigmoid_derivative(self.a1)

        self.W2 += self.lr * np.dot(self.a1.T, delta2)
        self.b2 += self.lr * delta2
        self.W1 += self.lr * np.dot(x.T, delta1)
        self.b1 += self.lr * delta1

        return 0.5 * np.sum(error ** 2)

# =====
# Класс MLP с функцией потерь BCE
# =====
class MLP_BCE:
    def __init__(self, lr, seed=None):
        if seed is not None:
            np.random.seed(seed)
        self.lr = lr
        self.W1 = np.random.randn(2, 2) * 0.5
        self.b1 = np.random.randn(1, 2) * 0.5
        self.W2 = np.random.randn(2, 1) * 0.5
        self.b2 = np.random.randn(1, 1) * 0.5

    def forward(self, x):
        self.z1 = np.dot(x, self.W1) + self.b1
        self.a1 = sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = sigmoid(self.z2)
        return self.a2

    def train_step(self, x, y_true):
        pred = self.forward(x)
        pred_clipped = np.clip(pred, 1e-15, 1 - 1e-15)
        error = y_true - pred

        delta2 = error
        delta1 = np.dot(delta2, self.W2.T) * sigmoid_derivative(self.a1)

        self.W2 += self.lr * np.dot(self.a1.T, delta2)
        self.b2 += self.lr * delta2

```

```

        self.W1 += self.lr * np.dot(x.T, delta1)
        self.b1 += self.lr * delta1

    bce_loss = -(
        y_true * np.log(pred_clipped)
        + (1 - y_true) * np.log(1 - pred_clipped)
    )
    return np.sum(bce_loss)

# =====
# Обучение модели
# =====
def train_model(model_class, target_error, title, seed=None):
    model = model_class(lr=LEARNING_RATE, seed=seed)
    history = []

    for epoch in range(MAX_EPOCHS):
        indices = np.arange(len(X_norm))
        np.random.shuffle(indices)

        epoch_error = 0.0
        for i in indices:
            epoch_error += model.train_step(X_norm[i:i + 1], y_norm[i:i + 1])

        history.append(epoch_error)

        if epoch_error <= target_error:
            return model, history, epoch + 1, True

    return model, history, MAX_EPOCHS, False

# =====
# Метрики
# =====
def calc_mae(model):
    mae = 0.0
    for i in range(len(X_norm)):
        pred_norm = model.forward(X_norm[i:i + 1])
        pred_real = denormalize(pred_norm)[0][0]
        mae += abs(pred_real - y_raw[i][0])
    return mae / len(X_norm)

def calc_accuracy(model):
    correct = 0
    for i in range(len(X_norm)):
        pred_norm = model.forward(X_norm[i:i + 1])[0][0]
        pred_real = denormalize(pred_norm)
        true_val = y_raw[i][0]

        pred_class = C0 if abs(pred_real - C0) < abs(pred_real - C1) else C1
        true_class = C0 if abs(true_val - C0) < abs(true_val - C1) else C1

        if pred_class == true_class:
            correct += 1

    return correct / len(X_norm) * 100

# =====
# Основной сценарий
# =====
print("=== Обучение эталонных моделей "
      "(Конфигурация А - MSE и Конфигурация В - BCE) ===")

model_mse, hist_mse, ep_mse, conv_mse = train_model(
    MLP_MSE, TARGET_ERROR_MSE, "MSE", seed=42
)
print(f"MSE: Эпох = {ep_mse}, Итоговая ошибка = {hist_mse[-1]:.5f}, "
      f"Сходимость = {conv_mse}")

model_bce, hist_bce, ep_bce, conv_bce = train_model(
    MLP_BCE, TARGET_ERROR_BCE, "BCE", seed=42
)
print(f"BCE: Эпох = {ep_bce}, Итоговая ошибка = {hist_bce[-1]:.5f}, "
      f"Сходимость = {conv_bce}")

print("\n=== Запуск серии из 5 экспериментов с разными seed ===")
seeds = [10, 42, 123, 777, 2024]
mse_epochs, bce_epochs = [], []
mse_converged_list, bce_converged_list = [], []

for s in seeds:
    _, _, e_mse, c_mse = train_model(MLP_MSE, TARGET_ERROR_MSE, "MSE", seed=s)
    _, _, e_bce, c_bce = train_model(MLP_BCE, TARGET_ERROR_BCE, "BCE", seed=s)

    mse_epochs.append(e_mse)
    bce_epochs.append(e_bce)
    mse_converged_list.append(c_mse)
    bce_converged_list.append(c_bce)

```

```

        status_mse = "✓" if c_mse else "X"
        status_bce = "✓" if c_bce else "X"
        print(f"Seed {s:4d}: MSE {status_mse} {e_mse} эпох, "
              f"BCE {status_bce} {e_bce} эпох.")

mae_mse = calc_mae(model_mse)
mae_bce = calc_mae(model_bce)
acc_mse = calc_accuracy(model_mse)
acc_bce = calc_accuracy(model_bce)

print(f"\n=== Итоговые метрики ===")
print(f"MSE: Accuracy = {acc_mse:.1f}%, MAE = {mae_mse:.4f}")
print(f"BCE: Accuracy = {acc_bce:.1f}%, MAE = {mae_bce:.4f}")

# =====
# График 1. Сходимость
# =====
print("\nОткрывается первый график "
      "(закройте окно, чтобы появился следующий)...")

plt.figure(figsize=(10, 6))
plt.plot(hist_mse, label='Конфигурация А (MSE)', color='blue', linewidth=2)
plt.plot(hist_bce, label='Конфигурация В (BCE)', color='red', linewidth=2)
plt.axhline(TARGET_ERROR_MSE, color='blue', linestyle=':', linewidth=1.5,
            label=f'Порог MSE ({TARGET_ERROR_MSE})')
plt.axhline(TARGET_ERROR_BCE, color='red', linestyle=':', linewidth=1.5,
            label=f'Порог BCE ({TARGET_ERROR_BCE})')
plt.title('Сходимость: Es от номера эпохи (представительный запуск)')
plt.xlabel('Эпоха')
plt.ylabel('Суммарная ошибка (Es)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, min(2000, MAX_EPOCHS))
plt.show()

# =====
# График 2. Разброс эпох по запускам
# =====
print("Открывается второй график "
      "(закройте окно, чтобы появился следующий)...")

x = np.arange(len(seeds))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 6))
bars_mse = ax.bar(x - width / 2, mse_epochs, width,
                  label='MSE', color='skyblue', edgecolor='black')
bars_bce = ax.bar(x + width / 2, bce_epochs, width,
                  label='BCE', color='lightcoral', edgecolor='black')

for bar, conv in zip(bars_mse, mse_converged_list):
    if not conv:
        bar.set_hatch('///')
        bar.set_label('MSE (не сошлась)')

for bar, conv in zip(bars_bce, bce_converged_list):
    if not conv:
        bar.set_hatch('///')
        bar.set_label('BCE (не сошлась)')

ax.set_title('Разброс числа эпох по 5 запускам\n'
              '(штриховка = не сошлась за MAX_EPOCHS)')
ax.set_xlabel('Номер запуска (seed)')
ax.set_ylabel('Эпохи')
ax.set_xticks(x)
ax.set_xticklabels([f'Run {i + 1}\n(seed={s})' for i, s in enumerate(seeds)])
ax.legend()
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# График 3. Разделяющие поверхности (>>> ИЗМЕНЕНО оформление)
# =====
print("Открывается третий график "
      "(закройте окно, чтобы появился следующий)...")

xx, yy = np.meshgrid(
    np.linspace(C0 - 2, C1 + 2, 300),
    np.linspace(C0 - 2, C1 + 2, 300),
)
grid_points = np.c_[xx.ravel(), yy.ravel()]
grid_norm = normalize(grid_points)

Z_mse = np.array([
    model_mse.forward(p.reshape(1, 2))[0][0] for p in grid_norm
]).reshape(xx.shape)

Z_bce = np.array([

```

```

        model_bce.forward(p.reshape(1, 2))[0][0] for p in grid_norm
    ]).reshape(xx.shape)

Z_mse_real = denormalize(Z_mse)
Z_bce_real = denormalize(Z_bce)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# >>> ИЗМЕНЕНО: другой фон - новая палитра, больше уровней,
#             полупрозрачность и контурные линии
cmap = plt.cm.coolwarm # было RdYlBu
levels = 100 # было 50

c1 = ax1.contourf(xx, yy, Z_mse_real, levels=levels,
                  cmap=cmap, alpha=0.85)
ax1.contour(xx, yy, Z_mse_real, levels=12,
            colors='k', linewidths=0.4, alpha=0.35)
ax1.scatter(X_raw[:, 0], X_raw[:, 1], c=y_raw.flatten(),
            cmap=cmap, edgecolors='white', linewidths=2,
            s=180, marker='o', label='Точки XOR', zorder=5)
ax1.set_title('Разделяющая поверхность (MSE)', fontsize=13)
ax1.set_xlabel('Вход A')
ax1.set_ylabel('Вход B')
ax1.legend(loc='upper right', framealpha=0.9)
fig.colorbar(c1, ax=ax1, label='Выход сети (шкала [c0; c1])')

c2 = ax2.contourf(xx, yy, Z_bce_real, levels=levels,
                  cmap=cmap, alpha=0.85)
ax2.contour(xx, yy, Z_bce_real, levels=12,
            colors='k', linewidths=0.4, alpha=0.35)
ax2.scatter(X_raw[:, 0], X_raw[:, 1], c=y_raw.flatten(),
            cmap=cmap, edgecolors='white', linewidths=2,
            s=180, marker='o', label='Точки XOR', zorder=5)
ax2.set_title('Разделяющая поверхность (BCE)', fontsize=13)
ax2.set_xlabel('Вход A')
ax2.set_ylabel('Вход B')
ax2.legend(loc='upper right', framealpha=0.9)
fig.colorbar(c2, ax=ax2, label='Выход сети (шкала [c0; c1])')

plt.suptitle('Визуализация разделяющей поверхности для обеих конфигураций',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

# =====
# График 4. Сравнение MAE
# =====
print("Открывается четвертый график "
      "(закройте окно, чтобы появился следующий)...")

plt.figure(figsize=(8, 6))
bars = plt.bar(
    ['Конфигурация A\n(MSE)', 'Конфигурация B\n(BCE)'],
    [mae_mse, mae_bce],
    color=['skyblue', 'lightcoral'],
    width=0.6,
    edgecolor='black',
)
plt.title('Сравнение точности восстановления шкалы\n'
          'Средняя абсолютная ошибка (MAE) в шкале [c0; c1]',
          fontsize=12, fontweight='bold')
plt.ylabel('MAE', fontsize=11)
plt.grid(axis='y', alpha=0.3)

for bar, val in zip(bars, [mae_mse, mae_bce]):
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.05,
             f'{val:.4f}', ha='center', va='bottom',
             fontsize=11, fontweight='bold')

plt.tight_layout()
plt.show()

# =====
# Режим функционирования
# =====
print("\n== РЕЖИМ ФУНКЦИОНИРОВАНИЯ (на базе конфигурации Б - BCE) ==")

# >>> ИЗМЕНЕНО: тестовые точки под новую шкалу [2; 10]
test_cases = [
    (2, 2), (2, 10), (10, 2), (10, 10),
    (6, 6), (4, 8), (7, 3),
]

print(f"\n{'Вход (A, B)':<15} | {'ŷ (0;1)':<10} | "
      f"{'ŷ исходная':<12} | {'Ближайший класс'}")
print("-" * 65)

for a, b in test_cases:
    x_in = normalize(np.array([a, b]))
    out_norm = model_bce.forward(x_in)[0][0]
    out_real = denormalize(out_norm)

```

```

dist_c0 = abs(out_real - C0)
dist_c1 = abs(out_real - C1)
closest = C0 if dist_c0 < dist_c1 else C1

print(f"({a:>4}, {b:>4}) | {out_norm:>8.4f} | "
      f"{out_real:>10.4f} | {closest}")

print("\n--- Интерактивный режим (введите 'exit' для выхода) ---")
print("Введите координаты A и B через пробел "
      f"(например: {int(C0)} {int(C1)})")

while True:
    try:
        user_input = input("\nВходные значения A и B: ")
        if user_input.lower() == 'exit':
            print("Завершение работы.")
            break

        a, b = map(float, user_input.split())

        x_in = normalize(np.array([[a, b]]))
        out_norm = model_bce.forward(x_in)[0][0]
        out_real = denormalize(out_norm)

        dist_c0 = abs(out_real - C0)
        dist_c1 = abs(out_real - C1)
        closest = C0 if dist_c0 < dist_c1 else C1

        print(f"Нормализованный выход      : {out_norm:.4f}")
        # >>> ИЗМЕНЕНО: шкала выводится динамически
        print(f"Выход в шкале [{int(C0)}; {int(C1)}]: {out_real:.4f}")
        print(f"Ближайший класс: {closest}")

    except ValueError:
        print("Ошибка ввода. Пожалуйста, введите два числа через пробел "
              "или 'exit' для выхода.")

```

Основные отличия от ЛР №1:

1. Два класса моделей: MLP_MSE (из ЛР №1) и MLP_BCE (новый)
2. Нормализация данных: $y_norm = \text{normalize}(y_raw)$
3. Разные функции потерь: MSE используется $0.5 \cdot (y - \hat{y})^2$, BCE использует $-[y \cdot \ln(\hat{y}) + (1 - y) \cdot \ln(1 - \hat{y})]$
4. Разные градиенты: MSE $\delta_2 = (y - \hat{y}) \cdot \hat{y}(1 - \hat{y})$, BCE $\delta_2 = (y - \hat{y})$
5. Разные пороги сходимости: TARGET_ERROR_MSE = 0.01, TARGET_ERROR_BCE = 0.05
6. Дополнительные метрики: функции calc_accuarcy() и calc_mae()
7. Режим функционирования: автоматическая демонстрация на 7 примерах + интерактивный ввод

Результат программы:

```

=== Обучение эталонных моделей (Конфигурация А - MSE и Конфигурация Б - BCE) ===
MSE: Эпох = 2092, Итоговая ошибка = 0.01000, Сходимость = True
BCE: Эпох = 756, Итоговая ошибка = 0.04997, Сходимость = True

=== Запуск серии из 5 экспериментов с разными seed ===
Seed 10: MSE ✓ 2114 эпох, BCE ✓ 703 эпох.
Seed 42: MSE ✓ 2092 эпох, BCE ✓ 756 эпох.
Seed 123: MSE ✗ 10000 эпох, BCE ✓ 671 эпох.
Seed 777: MSE ✓ 1610 эпох, BCE ✓ 647 эпох.
Seed 2024: MSE ✓ 1521 эпох, BCE ✓ 781 эпох.

=== Итоговые метрики ===
MSE: Accuarcy = 100.0%, MAE = 0.5628
BCE: Accuarcy = 100.0%, MAE = 0.0985

```

```

Входные значения A и B: 2 10
Нормализованный выход      : 0.9897
Выход в шкале [2; 10]: 9.9175
Ближайший класс: 10.0

```

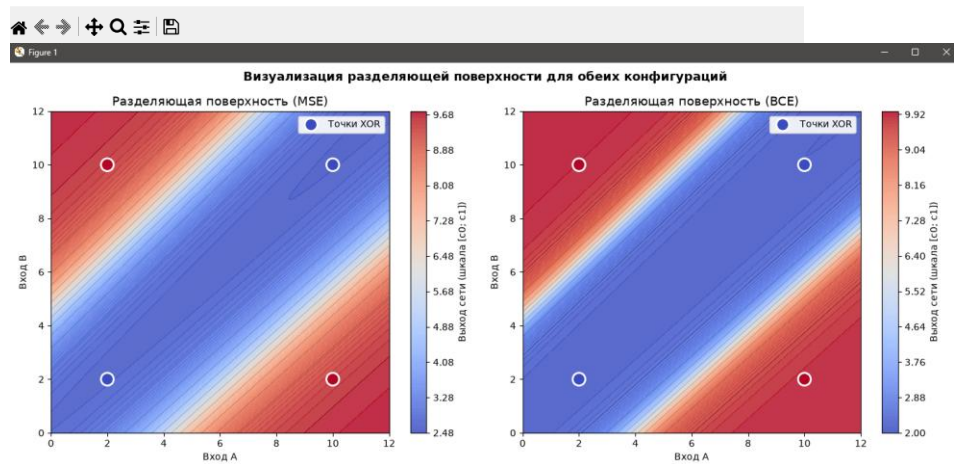
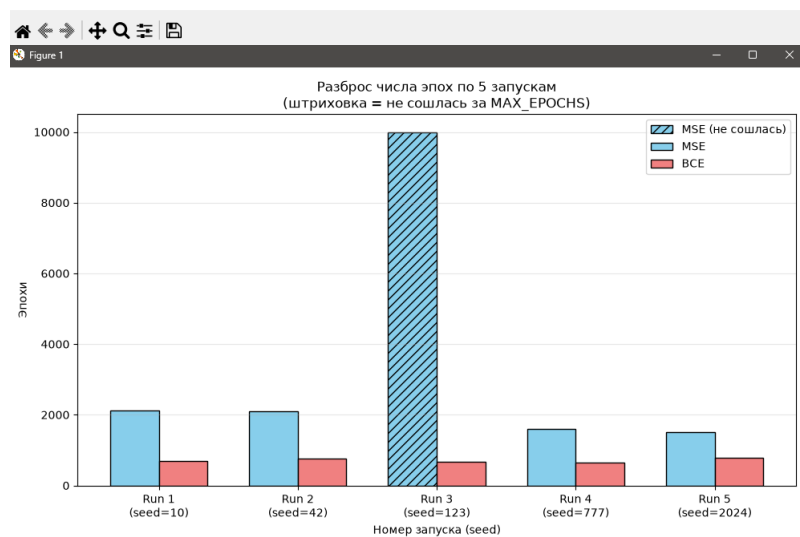
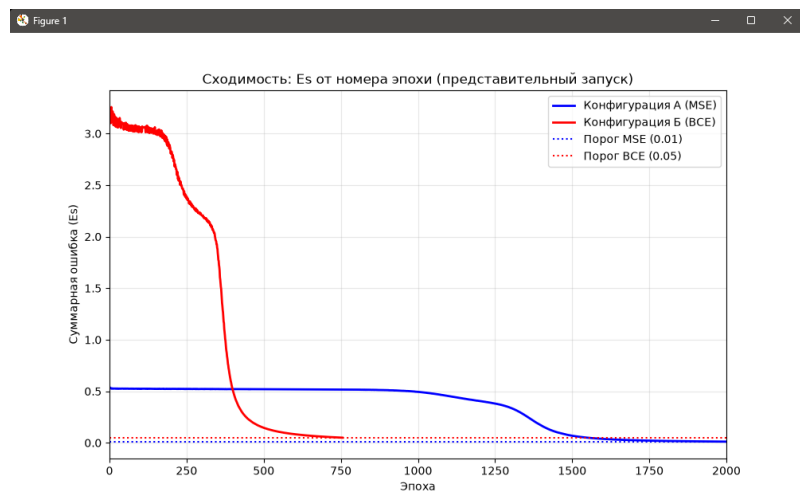
```

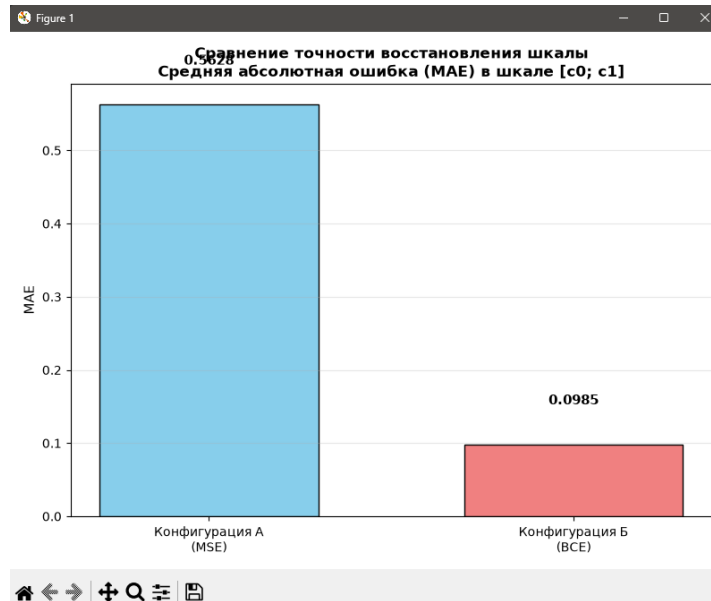
Входные значения A и B: exit
Завершение работы.

```

=== РЕЖИМ ФУНКЦИОНИРОВАНИЯ (на базе конфигурации Б - BCE) ===

Вход (A, B)	$\hat{y} (0;1)$	$\hat{y}$ исходная	Ближайший класс
(2, 2)	0.0118	2.0948	2.0
(2, 10)	0.9897	9.9175	10.0
(10, 2)	0.9831	9.8646	10.0
(10, 10)	0.0102	2.0813	2.0
(6, 6)	0.0109	2.0873	2.0
(4, 8)	0.2858	4.2861	2.0
(7, 3)	0.4332	5.4657	2.0





Вывод о проделанной работе:

1. Какая функция потерь обеспечила более быструю и устойчивую сходимость?

BCE показала преимущество: сошлась за 756 эпох против 2092 у MSE (в ~2.8 раза быстрее) и обеспечила 100% сходимость (5/5 запусков) против 80% у MSE. Это согласуется с теорией: градиент BCE+сигмоида упрощается до $(y - \hat{y})$, что исключает затухание градиента при больших ошибках.

2. Как нормализация повлияла на обучение?

Нормализация $c_0/c_1 \rightarrow 0/1$ критически важна для BCE, так как сигмоида выдаёт значения только в (0;1). Это упростило задачу оптимизации и повысило стабильность: BCE с нормализацией сошлась в 100% случаев, тогда как в ЛР №1 MSE без нормализации давала 80% сходимости и требовала больше эпох.

3. Точность восстановления шкалы и компромисс BCE/MSE

Парадокс: BCE дала MAE = 0.0985, а MSE — 0.5628 (в 5.7 раз хуже). BCE выдаёт более «уверенные» предсказания (~0.01 и ~0.99), которые после денормализации дают значения, близкие к 2 и 10. Компромисса нет — BCE в данном эксперименте превосходит MSE и по классификации (100% ассигасу у обеих), и по точности регрессии.

4. Устойчивость к инициализации весов

BCE: 5/5 запусков сошлись, разброс 647–781 эпох — высокая устойчивость.

MSE: 4/5 запусков сошлись, seed=123 не достиг порога за 10000 эпох (попал в локальный минимум). Разброс 1521–2114 эпох — средняя устойчивость.

5. Линейная неразделимость XOR

Подтвердилось. Независимо от кодирования классов (исходные 2/10 или нормализованные 0/1) задача XOR остаётся линейно неразделимой — однослойный персептрон не способен её решить. Это свойство самой задачи, а не способа кодирования.

6. Влияние типа функции потерь на необходимость многослойной архитектуры

Не влияет. Необходимость скрытого слоя определяется только топологией задачи (линейная неразделимость XOR). И MSE, и BCE требуют архитектуру 2-2-1 — ни одна из них не сможет обучить SLP на XOR.

7. Визуализация разделяющей поверхности

Обе конфигурации показывают характерную для XOR нелинейную границу в форме «седла» — пересечение двух полуплоскостей, формируемых скрытым слоем. Визуально поверхности похожи, но у BCE граница более резкая (контрастный переход), так как BCE даёт более уверенные предсказания. У MSE переход плавнее, что соответствует её меньшей уверенности в предсказаниях.

